

**THE UNIVERSITY OF AZAD JUMMAU & KASHMIR,
MUZAFFARABAD**

Name

Fatima Arshad

Roll No

2024-SE-24

Semester

04

Course Title

DBMS

Submitted By

Engr. Awais Rathore

Lab

10

1. Introduction

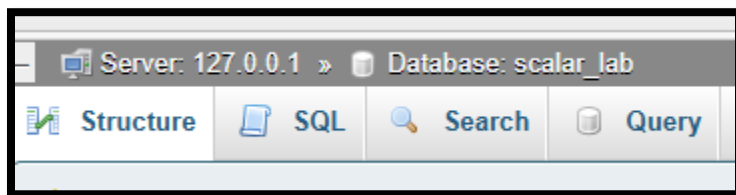
Scalar functions operate on one row at a time and return one value for each row. In this lab, string functions are used to clean, format, extract, and modify text data stored in the Customer and Product tables.

The **main** string functions used in this lab are:

- TRIM()
- UPPER()
- LOWER()
- CHAR_LENGTH()
- CONCAT()
- SUBSTRING()
- LEFT()
- REPLACE()
- LPAD()
- LOCATE()

➤ Create Database

```
CREATE DATABASE IF NOT EXISTS  
    scalar_lab;  
USE scalar_lab;
```



2. Create and Insert Tables (Customer , Product)

Table	Action	Rows	Type	Collation	Size	Overhead
☐ customer	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ product	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	16.0 KiB	-
2 tables Sum		0	InnoDB	utf8mb4_general_ci	32.0 KiB	0 B

SELECT * FROM `customer`

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: | Sort by key:

Extra options

		CustID	CustName	Email	City	Phone	JoinDate	DOB
☐	Edit Copy Delete	1	Ali Khan	ali.khan@MAIL.com	Lahore	0300-1112233	2022-01-15	1995-04-12
☐	Edit Copy Delete	2	Sara Iqbal	sara@example.com	Karachi	0301-4445566	2022-04-22	1998-11-20
☐	Edit Copy Delete	3	HAMZA RAZA	hamza@example.com	Lahore	0302-7778899	2023-02-10	1997-08-05
☐	Edit Copy Delete	4	Ayesha Noor	NULL	Islamabad	0303-1234567	2023-05-18	1999-02-14
☐	Edit Copy Delete	5	bilal ahmed	bilal@MAIL.COM	Karachi	0304-2345678	2023-09-01	2000-06-30
☐	Edit Copy Delete	6	Fatima Sheikh	fatima@example.com	NULL	0305-3456789	2024-01-12	1996-10-25
☐	Edit Copy Delete	7	Usman Tariq	usman@example.com	Lahore	NULL	2024-06-30	2001-03-18
☐	Edit Copy Delete	8	Maira Javed	maira@example.com	Islamabad	0307-5678901	2024-08-25	1994-12-09

☐ Check all | 8/8 selected | Edit Copy Delete Export

SELECT * FROM `product`

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

		ProdID	ProdName	Category	Price	StockQty	LaunchDate
☐	Edit Copy Delete	101	Laptop Pro 15	Electronics	185000.00	12	2023-03-10
☐	Edit Copy Delete	102	Wireless Mouse	Electronics	2500.00	50	2022-07-22
☐	Edit Copy Delete	103	USB-C Cable	Electronics	800.00	100	2021-11-05
☐	Edit Copy Delete	104	Office Chair	Furniture	18500.00	8	2023-01-15
☐	Edit Copy Delete	105	Standing Desk	Furniture	45000.50	5	2024-02-28
☐	Edit Copy Delete	106	Notebook A4	Stationery	350.00	200	2020-04-01
☐	Edit Copy Delete	107	Ballpoint Pen 10pk	Stationery	450.00	150	2020-04-01
☐	Edit Copy Delete	108	Coffee Beans 1kg	Grocery	1899.99	30	2023-09-20
☐	Edit Copy Delete	109	Green Tea Box	Grocery	650.00	45	2022-12-12
☐	Edit Copy Delete	110	Bluetooth Speaker	Electronics	7500.00	18	2024-05-18

3. Part A — String Functions

Task A1 — Remove Leading and Trailing Spaces

Objective:

List each customer's name with leading and trailing spaces removed while also displaying the original name.

SQL Query:

```
SELECT
    CustID,
    CustName AS OriginalName,
    TRIM(CustName) AS CleanedName
FROM Customer;
```

Functions Used: TRIM()

Description: The TRIM() function removes unnecessary spaces from the beginning and end of the customer's name.

✓ Showing rows 0 - 7 (8 total, Query took 0.0004 seconds.)

```
SELECT CustID, CustName AS OriginalName, TRIM(CustName) AS CleanedName FROM Customer;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▼ Filter rows: Sort by ke

Extra options

				CustID	OriginalName	CleanedName
☐	Edit	Copy	Delete	1	Ali Khan	Ali Khan
☐	Edit	Copy	Delete	2	Sara Iqbal	Sara Iqbal
☐	Edit	Copy	Delete	3	HAMZA RAZA	HAMZA RAZA
☐	Edit	Copy	Delete	4	Ayesha Noor	Ayesha Noor
☐	Edit	Copy	Delete	5	bilal ahmed	bilal ahmed
☐	Edit	Copy	Delete	6	Fatima Sheikh	Fatima Sheikh
☐	Edit	Copy	Delete	7	Usman Tariq	Usman Tariq
☐	Edit	Copy	Delete	8	Maira Javed	Maira Javed

Task A2 — Convert Names to Uppercase and Lowercase

Objective:

Display every customer's name in uppercase and lowercase.

SQL Query:

```
SELECT
    CustID,
    UPPER(CustName) AS UpperName,
    LOWER(CustName) AS
    LowerName
FROM Customer;
```

Functions Used: UPPER(), LOWER()

Description: UPPER() converts all letters to uppercase, while LOWER() converts all letters to lowercase.

SELECT CustID, UPPER(CustName) AS UpperName, LOWER(CustName) AS LowerName FROM Customer;

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▼ Filter rows: Sort by key:

Extra options

				CustID	UpperName	LowerName
☐		Edit		Copy		Delete
1	ALI KHAN	ali khan				
☐		Edit		Copy		Delete
2	SARA IQBAL	sara iqbal				
☐		Edit		Copy		Delete
3	HAMZA RAZA	hamza raza				
☐		Edit		Copy		Delete
4	AYESHA NOOR	ayesha noor				
☐		Edit		Copy		Delete
5	BILAL AHMED	bilal ahmed				
☐		Edit		Copy		Delete
6	FATIMA SHEIKH	fatima sheikh				
☐		Edit		Copy		Delete
7	USMAN TARIQ	usman tariq				
☐		Edit		Copy		Delete
8	MAIRA JAVED	maira javed				

Task A3 — Trim Names and Count Characters

Objective:

Display each customer's trimmed name and the number of characters in it.

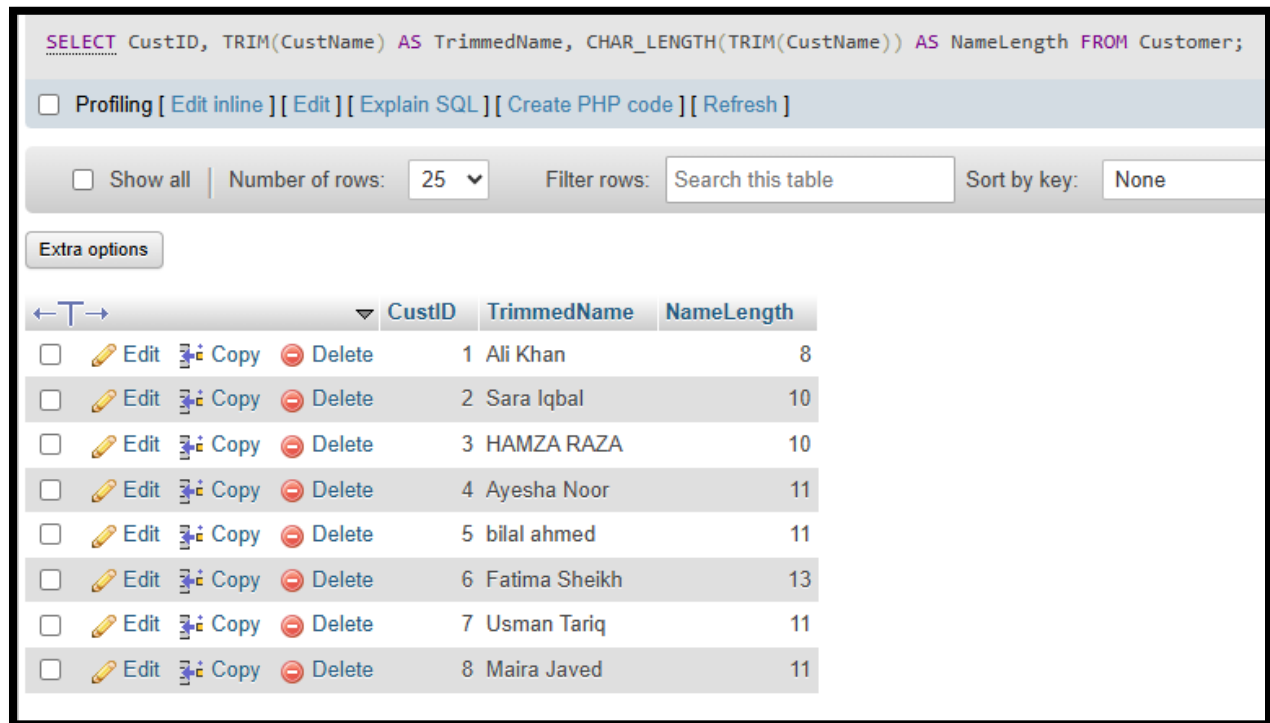
SQL Query:

```
SELECT
    CustID,
    TRIM(CustName) AS
    TrimmedName,

    CHAR_LENGTH(TRIM(CustName))
    AS NameLength
FROM Customer;
```

Functions Used: TRIM(), CHAR_LENGTH()

Description: First, TRIM() removes unnecessary spaces. Then CHAR_LENGTH() counts the characters in the cleaned name.



```
SELECT CustID, TRIM(CustName) AS TrimmedName, CHAR_LENGTH(TRIM(CustName)) AS NameLength FROM Customer;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				CustID	TrimmedName	NameLength
☐				1	Ali Khan	8
☐				2	Sara Iqbal	10
☐				3	HAMZA RAZA	10
☐				4	Ayesha Noor	11
☐				5	bilal ahmed	11
☐				6	Fatima Sheikh	13
☐				7	Usman Tariq	11
☐				8	Maira Javed	11

Task A4 — Create a Customer Greeting

Objective:

Build a greeting for every customer using their trimmed name.

SQL Query:

```
SELECT
    CustID,
    CONCAT('Dear ',
    TRIM(CustName), ', welcome!') AS
    Greeting
FROM Customer;
```

Functions Used: CONCAT(), TRIM()

Description: CONCAT() combines multiple pieces of text into one string.

Example Output:

Dear Ali Khan, welcome!

```
SELECT CustID, CONCAT('Dear ', TRIM(CustName), ', welcome!') AS Greeting FROM Customer;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▼ Filter rows: Sort by key:

Extra options

					CustID	Greeting	
☐		Edit		Copy		Delete	1 Dear Ali Khan, welcome!
☐		Edit		Copy		Delete	2 Dear Sara Iqbal, welcome!
☐		Edit		Copy		Delete	3 Dear HAMZA RAZA, welcome!
☐		Edit		Copy		Delete	4 Dear Ayesha Noor, welcome!
☐		Edit		Copy		Delete	5 Dear bilal ahmed, welcome!
☐		Edit		Copy		Delete	6 Dear Fatima Sheikh, welcome!
☐		Edit		Copy		Delete	7 Dear Usman Tariq, welcome!
☐		Edit		Copy		Delete	8 Dear Maira Javed, welcome!

Task A5 — Extract Username from Email

Objective:

For customers who have an email address, extract the part before the @ symbol.

SQL Query:

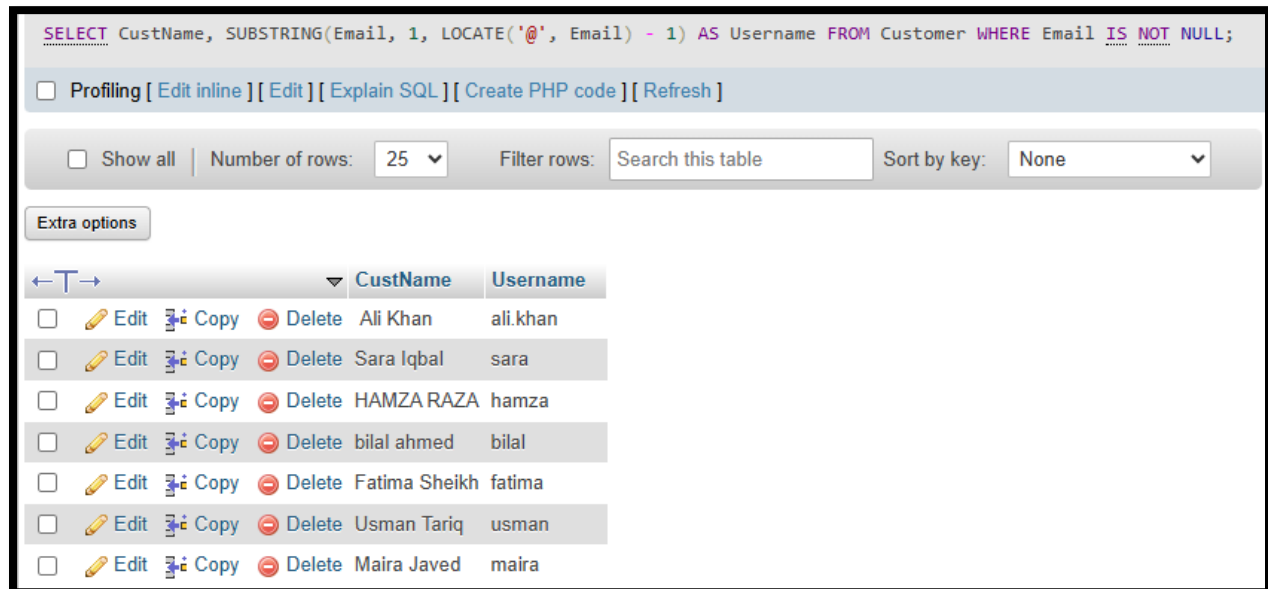
```
SELECT
    CustName,
    SUBSTRING(Email, 1,
LOCATE('@', Email) - 1) AS
Username
FROM Customer
WHERE Email IS NOT NULL;
```

Functions Used: SUBSTRING(), LOCATE()

Description: LOCATE() finds the position of @, and SUBSTRING() extracts everything before it.

Example: ali.khan@MAIL.com

Result: ali.khan



The screenshot shows a SQL query execution interface. At the top, the SQL query is displayed: `SELECT CustName, SUBSTRING(Email, 1, LOCATE('@', Email) - 1) AS Username FROM Customer WHERE Email IS NOT NULL;`. Below the query, there are options for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. A control bar includes a checkbox for 'Show all', a 'Number of rows' dropdown set to 25, a 'Filter rows' search box, and a 'Sort by key' dropdown set to None. An 'Extra options' button is also present. The main area displays a table with two columns: 'CustName' and 'Username'. Each row includes checkboxes for Edit, Copy, and Delete actions. The data rows are as follows:

	CustName	Username
☐	Ali Khan	ali.khan
☐	Sara Iqbal	sara
☐	HAMZA RAZA	hamza
☐	bilal ahmed	bilal
☐	Fatima Sheikh	fatima
☐	Usman Tariq	usman
☐	Maira Javed	maira

Task A6 — Extract Email Domain

Objective:

For customers who have an email address, extract everything after the @ symbol.

SQL Query:

```
SELECT
    CustName,
    SUBSTRING(Email,
LOCATE('@', Email) + 1) AS
Domain
FROM Customer
WHERE Email IS NOT NULL;
```

Functions Used: SUBSTRING(), LOCATE()

Description: The query finds the @ symbol and extracts the text after it.

Example: ali.khan@MAIL.com

Result: MAIL.com

```
SELECT CustName, SUBSTRING(Email, LOCATE('@', Email) + 1) AS Domain FROM Customer WHERE Email IS NOT NULL;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

					CustName	Domain
☐	Edit	Copy	Delete		Ali Khan	MAIL.com
☐	Edit	Copy	Delete		Sara Iqbal	example.com
☐	Edit	Copy	Delete		HAMZA RAZA	example.com
☐	Edit	Copy	Delete		bilal ahmed	MAIL.COM
☐	Edit	Copy	Delete		Fatima Sheikh	example.com
☐	Edit	Copy	Delete		Usman Tariq	example.com
☐	Edit	Copy	Delete		Maira Javed	example.com

Task A7 — Display First Three Characters

Objective:

Show only the first three characters of each customer's name.

SQL Query:

```
SELECT
    CustID,
    LEFT(TRIM(CustName), 3) AS
    First3Characters
FROM Customer;
```

Functions Used: LEFT(), TRIM()

Description: LEFT() returns the specified number of characters from the beginning of a string.

✓ Showing rows 0 - 7 (8 total, Query took 0.0004 seconds.)

```
SELECT CustID, LEFT(TRIM(CustName), 3) AS First3Characters FROM Customer;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: Filter rows: Sort by

Extra options

					CustID	First3Characters		
☐		Edit		Copy		Delete	1	Ali
☐		Edit		Copy		Delete	2	Sar
☐		Edit		Copy		Delete	3	HAM
☐		Edit		Copy		Delete	4	Aye
☐		Edit		Copy		Delete	5	bil
☐		Edit		Copy		Delete	6	Fat
☐		Edit		Copy		Delete	7	Usm
☐		Edit		Copy		Delete	8	Mai

Task A8 — Mask Customer Phone Numbers

Objective:

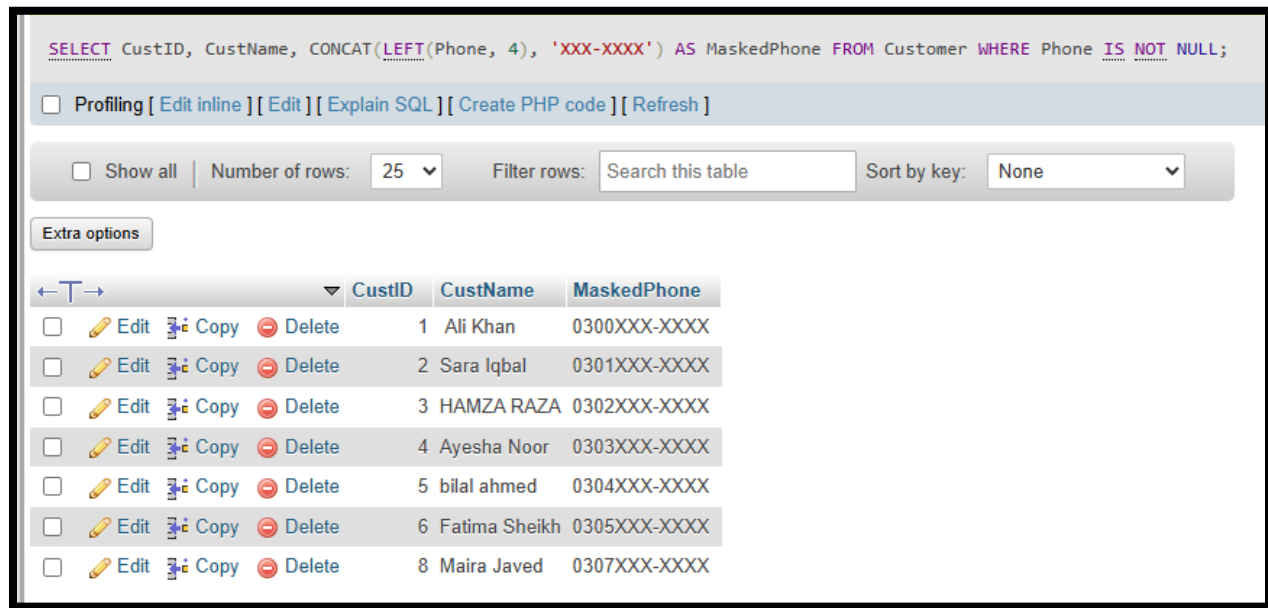
Display the first four characters of each phone number followed by XXX-XXXX. Customers without phone numbers should be skipped.

SQL Query:

```
SELECT
    CustID,
    CustName,
    CONCAT(LEFT(Phone, 4),
    'XXX-XXXX') AS MaskedPhone
FROM Customer
WHERE Phone IS NOT NULL;
```

Functions Used: LEFT(), CONCAT()

Description: The first four characters of the phone number are displayed, while the remaining portion is replaced with XXX-XXXX.



The screenshot shows a database interface with a SQL query and its results. The query is: `SELECT CustID, CustName, CONCAT(LEFT(Phone, 4), 'XXX-XXXX') AS MaskedPhone FROM Customer WHERE Phone IS NOT NULL;`. Below the query, there are options for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. A control bar shows 'Show all', 'Number of rows: 25', 'Filter rows: Search this table', and 'Sort by key: None'. An 'Extra options' button is also present. The results table has columns CustID, CustName, and MaskedPhone, with 8 rows of data.

	CustID	CustName	MaskedPhone
☐ Edit Copy Delete	1	Ali Khan	0300XXX-XXXX
☐ Edit Copy Delete	2	Sara Iqbal	0301XXX-XXXX
☐ Edit Copy Delete	3	HAMZA RAZA	0302XXX-XXXX
☐ Edit Copy Delete	4	Ayesha Noor	0303XXX-XXXX
☐ Edit Copy Delete	5	bilal ahmed	0304XXX-XXXX
☐ Edit Copy Delete	6	Fatima Sheikh	0305XXX-XXXX
☐ Edit Copy Delete	8	Maira Javed	0307XXX-XXXX

Task A9 — Replace Spaces with Hyphens

Objective:

Replace all spaces in product names with hyphens.

SQL Query:

```
SELECT
    ProdID,
    REPLACE(ProdName, ' ', '-')
AS SlugName
FROM Product;
```

Function Used: REPLACE()

Description: REPLACE() replaces every occurrence of one character or string with another.

Example: Laptop Pro 15

becomes:

Laptop-Pro-15

✓ Showing rows 0 - 9 (10 total, Query took 0.0004 seconds.)

```
SELECT ProdID, REPLACE(ProdName, ' ', '-') AS SlugName FROM Product;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort

Extra options

	ProdID	SlugName
☐ Edit Copy Delete	101	Laptop-Pro-15
☐ Edit Copy Delete	102	Wireless-Mouse
☐ Edit Copy Delete	103	USB-C-Cable
☐ Edit Copy Delete	104	Office-Chair
☐ Edit Copy Delete	105	Standing-Desk
☐ Edit Copy Delete	106	Notebook-A4
☐ Edit Copy Delete	107	Ballpoint-Pen-10pk
☐ Edit Copy Delete	108	Coffee-Beans-1kg
☐ Edit Copy Delete	109	Green-Tea-Box
☐ Edit Copy Delete	110	Bluetooth-Speaker

Console

Task A10 — Pad Product IDs with Zeros

Objective:

Display each product ID as a five-digit value by adding leading zeros.

SQL Query:

```
SELECT
    ProdID,
    LPAD(ProdID, 5, '0') AS
    PaddedProdID
FROM Product;
```

Function Used: LPAD()

Description: LPAD() adds characters to the left side of a value until it reaches the specified length.

Example: 101 → 00101

The screenshot shows a SQL query execution interface. At the top, the SQL query is: `SELECT ProdID, LPAD(ProdID, 5, '0') AS PaddedProdID FROM Product;`. Below the query, there are options for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. A table view is selected, showing 25 rows. The table has two columns: ProdID and PaddedProdID. The data shows ProdID values from 101 to 110, with PaddedProdID values being the ProdID padded with zeros to a length of 5. For example, 101 becomes 00101, and 110 becomes 00110. Each row has Edit, Copy, and Delete icons. A Console tab is visible at the bottom left.

ProdID	PaddedProdID
101	00101
102	00102
103	00103
104	00104
105	00105
106	00106
107	00107
108	00108
109	00109
110	00110

Task A11 — Find Products Containing "Pro"

Objective:

Display product names that contain the word Pro and show the position where Pro starts.

SQL Query:

```
SELECT
    ProdID,
    ProdName,
    LOCATE('Pro', ProdName)
AS ProPosition
FROM Product
WHERE LOCATE('Pro',
ProdName) > 0;
```

Function Used: LOCATE()

Description: LOCATE() returns the starting position of the specified text. If the text is not found, it returns 0.

The screenshot displays a database management system interface. At the top, a green status bar indicates "Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)". Below this, the SQL query is shown: `SELECT ProdID, ProdName, LOCATE('Pro', ProdName) AS ProPosition FROM Product WHERE LOCATE('Pro', ProdName) > 0;`. A toolbar offers options like "Profiling", "Edit inline", "Edit", "Explain SQL", "Create PHP code", and "Refresh". Below the toolbar, there are controls for "Show all", "Number of rows" (set to 25), and a "Filter rows" search box. An "Extra options" button is also present. The main area shows a table with three columns: "ProdID", "ProdName", and "ProPosition". The first row contains the values "101", "Laptop Pro 15", and "8". Below the table, there are more controls for "Show all", "Number of rows" (25), and "Filter rows". At the bottom, a "Query results operations" section includes buttons for "Print", "Copy to clipboard", "Export", "Display chart", and "Create view". A "Bookmark this SQL query" button is located at the very bottom.

ProdID	ProdName	ProPosition
101	Laptop Pro 15	8

Task A12 — Extract Customer First Name

Objective:

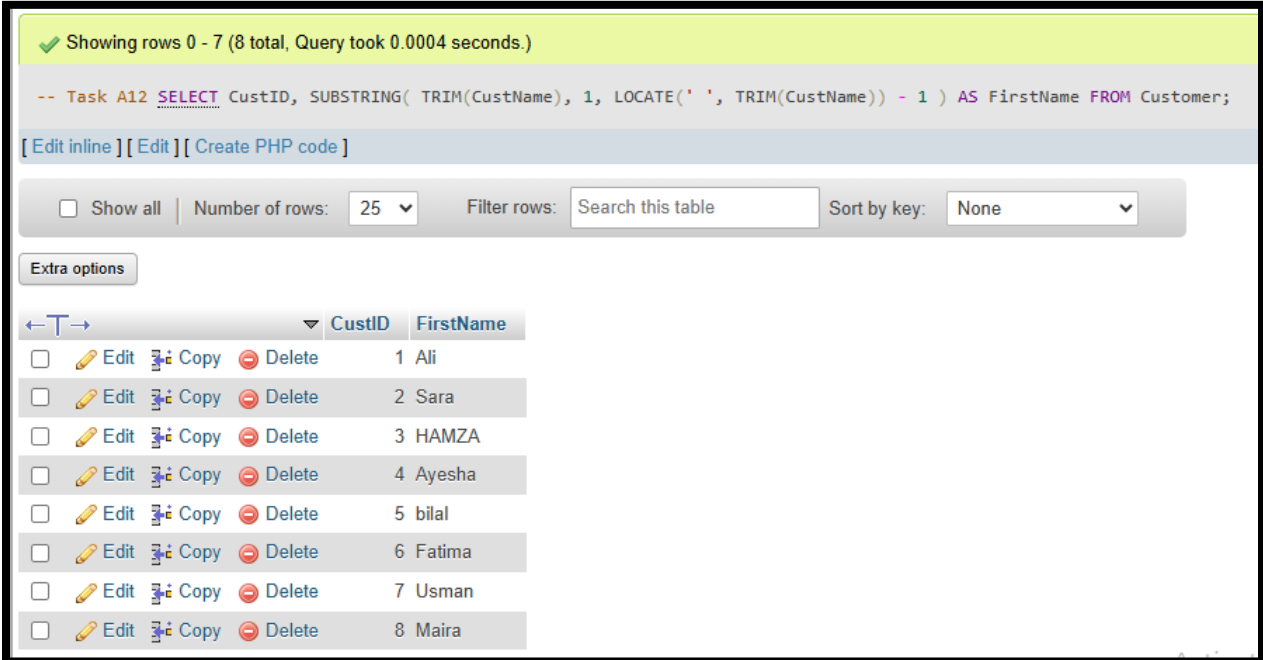
Display only the first name of every customer.

SQL Query:

```
SELECT
CustID,
SUBSTRING(
    TRIM(CustName),
    1,
    LOCATE(' ',
    TRIM(CustName)) - 1
) AS FirstName
FROM Customer;
```

Functions Used: TRIM(), LOCATE(), SUBSTRING()

Description: First, TRIM() removes extra spaces. Then LOCATE() finds the first space, and SUBSTRING() extracts the text before that space.



Showing rows 0 - 7 (8 total, Query took 0.0004 seconds.)

```
-- Task A12 SELECT CustID, SUBSTRING( TRIM(CustName), 1, LOCATE(' ', TRIM(CustName)) - 1 ) AS FirstName FROM Customer;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				CustID	FirstName
☐	Edit	Copy	Delete	1	Ali
☐	Edit	Copy	Delete	2	Sara
☐	Edit	Copy	Delete	3	HAMZA
☐	Edit	Copy	Delete	4	Ayesha
☐	Edit	Copy	Delete	5	bilal
☐	Edit	Copy	Delete	6	Fatima
☐	Edit	Copy	Delete	7	Usman
☐	Edit	Copy	Delete	8	Maira

Conclusion

In this lab, different MySQL string functions were applied to customer and product data. The queries demonstrated how to remove unnecessary spaces, change letter cases, join text, extract usernames and domains, replace characters, mask phone numbers, pad IDs, and locate specific text within strings.
